MEDAC
Instituto Oficial de Formación Profesional

Desarrollo de Aplicación Móvil de Inicio de Sesión (LoginApp)

Autor: Miguel Ángel Toro Chaves

Fecha: 25 oct 2026

Enlace: https://www.figma.com/design/BBMmtelf0IpF5QviS3qvGe/Reto-2---Wireframes?node-id=24-7&m=dev

1. Introducción y Objetivos

El presente proyecto consiste en el desarrollo de una aplicación móvil de inicio de sesión denominada LoginApp, diseñada para el módulo de Programación del centro MEDAC. El objetivo principal es ofrecer una interfaz fluida, accesible y moderna para la autenticación de usuarios.

Para alcanzar este propósito, se han establecido los siguientes objetivos técnicos:

- Diseño responsivo: Uso de *ConstraintLayout* para garantizar una disposición adaptable en diversos tamaños de pantalla.
- Acceso a vistas seguro: Implementación de *ViewBinding* para interactuar con los elementos gráficos eliminando riesgos de punteros nulos.

- Navegación fluida: Uso de *Intents* explícitos e implícitos para redirigir entre actividades y servicios de autenticación externos.

2. Diseño y Prototipado

La interfaz de usuario se ha estructurado con componentes nativos optimizados, priorizando la claridad y la accesibilidad en cada pantalla.

Entre los aspectos de diseño y recursos destacados se encuentran:

- Gestión de colores (colors.xml): Definición de una paleta cromática coherente con la identidad institucional, garantizando un contraste óptimo.
- Internacionalización (strings.xml): Centralización de los textos para facilitar el mantenimiento y la futura localización de la aplicación.
- Iconos adaptativos (drawable): Integración de recursos gráficos vectoriales y adaptativos para proveedores externos como Gmail, iCloud y Outlook.

3. Implementación Técnica y Código Clave

A continuación se detallan los extractos más relevantes de la arquitectura de la aplicación, incluyendo el diseño XML de la interfaz y la lógica de validación e interacción en Java.

Interfaz de Usuario (activity_login.xml)

XML

```
    <ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:app="http://schemas.android.com/apk/res-auto"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <EditText
            android:id="@+id/etEmail"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:hint="@string/hint_email"
            android:inputType="textEmailAddress"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent"/>

        <Button
            android:id="@+id/btnLogin"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:text="@string/btn_login"
            app:layout_constraintTop_toBottomOf="@id/etEmail"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent"/>
    </ConstraintLayout>
```

Lógica Principal (LoginActivity.java)

Java

```
    public class LoginActivity extends AppCompatActivity {
        private ActivityLoginBinding binding;
```

```
        @Override
        protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            binding = ActivityLoginBinding.inflate(getLayoutInflater());
            setContentView(binding.getRoot());

            binding.btnLogin.setOnClickListener(v -> {
                String email = binding.etEmail.getText().toString();
                if (isValidEmail(email)) {
                    Intent intent = new Intent(this, HomeActivity.class);
                    startActivity(intent);
                } else {
                    binding.etEmail.setError("Correo no válido");
                }
            });
        }
    }
```

4. Pruebas y Conclusiones

Las pruebas funcionales se llevaron a cabo utilizando el emulador oficial de Android Studio en diversas configuraciones de pantalla y versiones de API. Se verificó el correcto funcionamiento de las validaciones de entrada, la respuesta táctil de los botones y la transición fluida entre actividades mediante *Intents*.

Como valoración personal, el desarrollo de LoginApp ha permitido consolidar conceptos clave de programación orientada a componentes móviles, el uso eficiente de arquitecturas desacopladas mediante ViewBinding y las mejores prácticas en el diseño de interfaces de usuario.

1. ¿Qué decisión del análisis o de la preparación en Figma te ha resultado más difícil? ¿Por qué?

La decisión más compleja durante la fase de diseño en Figma y su posterior adaptación a Android Studio fue estructurar las restricciones (*constraints*) del `ConstraintLayout` para garantizar que la interfaz fuera completamente responsive. Mantener el espaciado correcto de los campos de texto (`EditText`) y alinear el contenedor horizontal de los iconos sociales (`LinearLayout`) sin que se deformasen en diferentes tamaños de pantalla exigió calcular con precisión los márgenes en dp y los porcentajes de anclaje.

2. Has pasado tus colores y fuentes a recursos. ¿Qué tendrías que cambiar si mañana el cliente modifica el color principal? ¿Y si no hubieras usado recursos?

- Utilizando recursos (`colors.xml`): Únicamente tendría que modificar el valor hexadecimal del color en el archivo centralizado `colors.xml`. La actualización se aplicaría de forma automática y global en todas las vistas, botones y elementos de la aplicación que hagan referencia a dicho recurso.
- Sin haber usado recursos: Habría tenido que buscar manualmente línea por línea en cada archivo de diseño XML (`activity_login.xml`, `activity_main.xml`, etc.) cada uno de los atributos `android:background` o `android:textColor` donde se hubiera escrito el color a mano, lo que incrementa drásticamente el riesgo de errores y de dejar pantallas sin actualizar.

3. ¿Cuánto ocupaba tu imagen en PNG y cuánto en WebP? ¿Qué conclusión sacas?

El archivo de imagen en formato PNG original registraba un peso superior, mientras que al convertirlo y optimizarlo a formato WebP (`img_header.webp`), el tamaño del

archivo se redujo de manera notable (habitualmente entre un 25% y un 35% menos) sin perder calidad visual.

- Conclusión: El uso del formato WebP es altamente recomendable en el desarrollo Android moderno. Permite aligerar el peso total del archivo APK y optimiza el consumo de memoria RAM y almacenamiento del dispositivo, mejorando el rendimiento general de la aplicación sin sacrificar la fidelidad gráfica del diseño.

Capturas:



